

IT UNIVERSITY OF COPENHAGEN

FIRST YEAR PROJECT 3

Natural Language Processing

Authors

Viggo U. GASCOU
Frida N. LAUSTSEN
Magnus S. JAHN

Authors

Marie H. PETERSEN
Gustav KRISTENSEN

Supervisor

Christian HARDMEIER

June 3th, 2022

1 Introduction

The development in the field of Natural Language Processing (NLP) has been on a steady incline since the mid twentieth century. A field initially focused on translating text from one language to another has now evolved into an industry and research field devoted to solving much more complex problems. Recent developments in the field include pattern recognition in health records to help predict unnoticed signs of disease, Amazon’s Alexa, as well as chat-bots as a supplement to customer support.

The expansion of social media in the last decade has lead to the ability for anyone to express their thoughts and feelings, as well as share their life with anyone in the world. For social media companies, their stakeholders, and researchers, the ability to interpret patterns and behaviours of users is of huge interest. With the evolution in NLP, the capability to understand the intentions and sentiments of users have vastly improved, resulting in an ability to automatically analyse complex linguistic phenomena such as irony and sarcasm (Van Hee et al., 2018).

In this project, our main goals were the development of an NLP model that could perform binary classification of irony in tweets, as well as a model to predict stances (positive, negative, neutral) on the topics abortion, atheism, feminism, climate change and Hillary Clinton. Our approach was supervised feature learning, using low level features. Particularly, using an SGD classifier model as well as an MLP classifier (a neural network classifier) on a bag of words.

2 Data and Preprocessing

We were provided with seven different data sets from the TweetEval repository, consisting of three data sets for the binary classification task, which were ‘Irony Detection’, ‘Hate Speech detection’ and ‘Offensive Language Detection’. Furthermore, the data sets consisted of four multiclass classification tasks, namely ‘Emotion Recognition’, ‘Emoji Prediction’, ‘Sentiment Analysis’ and ‘Stance Detection’. Each of the seven data sets additionally contained a (`mapping.txt`) file with the different classes as well as: (`test_labels.txt`), (`test_text.txt`), (`train_label.txt`), (`train_text.txt`), (`val_labels.txt`) and (`val_text.txt`).

For the two-class (binary) classification problem, we chose to work with ‘Irony Detection’. Here the goal was to predict whether a tweet was ironic or not. The data set consisted of 4601 tweets, constructed by searching for #irony, #sarcasm and #not on Twitter.

For the multi-class classification problem we chose to work with ‘Stance Detection’, with the task of determining whether a tweet was in favour of, against or neutral towards a specific topic. In this case, the targets are either abortion, atheism, climate change, feminism or Hillary Clinton. The data sets consisted of tweets paired with a target/topic of interest. The tweets and topics were selected such that they would both be understandable to a wide number of people in the US, and were subsequently annotated for stance by respondents living in the US. These data sets consisted of a total of 4163 tweets with all topics combined.

For the preprocessing procedure of the data sets, we inspected the training data set and implemented a tokeniser to split the tweets into meaningful tokens. Given that tweets do not necessarily follow orthographic conventions, it is essential that the tokeniser detects expressions that are common in tweets. For this reason our tokeniser detects and tokenises hashtags, emojis, dates, and timestamps, and keeps such instances intact. Proper nouns consisting of more than one word, are tokenised as one token. This was implemented by detecting consecutive words starting with uppercase letters. As a result of this decision there were some unwanted tokenisations. For instance, sentences written in all caps would be recognised as proper nouns and thus tokenised as one token.

Furthermore, we decided to tokenise "w/" and "w/o" as their corresponding words "with" and "without", in order to preserve the meaning of the words. We chose to keep quotation marks around words, as they can supply additional meaning to a word - for example one might put quotation marks around a word that was sarcastic. All other instances are split on white space, and single punctuation marks are discarded. Occurrences of 2 or more punctuation marks in a row are tokenised as one token.

Tokeniser comparison

After the tokenisation of our data, we ran the tokeniser over a portion of the training data set that, we did not hold out for the tokeniser evaluation, and compared the output with the baseline tokenisation we got from the social media tokeniser in the NLTK library (`nltk.tokenize.TweetTokenizer`).

When comparing the two tokenisers, we were interested in seeing how many unique tokens one method picked up that the other did not. Moreover, we figured that many of the unique tokens that one tokeniser picked up on, might just appear once or twice in the whole corpus. Therefore, we investigated how many unique words one tokeniser picked up on more than two times.

The tokenisers were tested on the Hillary stance training set. Our tokeniser found 13 unique tokens appearing more than twice in the corpus that the NLTK tokeniser did not pick up on. Meanwhile, the NLTK tokeniser found 37 unique tokens appearing more than twice that our tokeniser did not pick up on. The main differences in the most frequent tokens in the two tokenisers were (1) the NLTK tokeniser picked up on symbols such as "!", ".", "?" while ours intentionally did not. (2) Our initial idea was to pick up on proper names such as "New York" and "Hillary Clinton" and also to extract meaning from things such as ellipses or multiple exclamation marks. This is reflected in the most frequent tokens unique to our tokeniser. We later modified our preprocessing method to convert all letters in the corpus to lowercase, which made our decision to group proper names to one token redundant. The motivation for doing this is described later.

We explored the tokenised data sets and computed data statistics (Table 1).

Dataset	Corpus size	Vocabulary size	Type/token ratio
Binary			
Irony	38 944	10 168	0.261
Multiclass			
Abortion	4714	1727	0.366
Atheism	3816	1578	0.414
Climate	2763	1290	0.467
Feminist	4945	1832	0.370
Hillary	4822	1793	0.372
Multiclass Total	21 060	8220	

Table 1: Data Statistics

Additionally, we plotted the word frequencies by their rank, in order to evaluate if the corpora follows Zipf's law (Figure 1). The plots are log-log scatterplots, and in this case a straight line would be a perfect representation of Zipf's law. Our plots are more or less straight lines, so we say that the corpora do follow Zipf's law.

Since many tokens have the same meaning, but are represented as different types, it was convenient for us to normalise the tokens. Therefore, we made all text lower-case, to normalise tweets written in all caps, and expanded clitic contractions marked by apostrophes by using the `contractions` library.

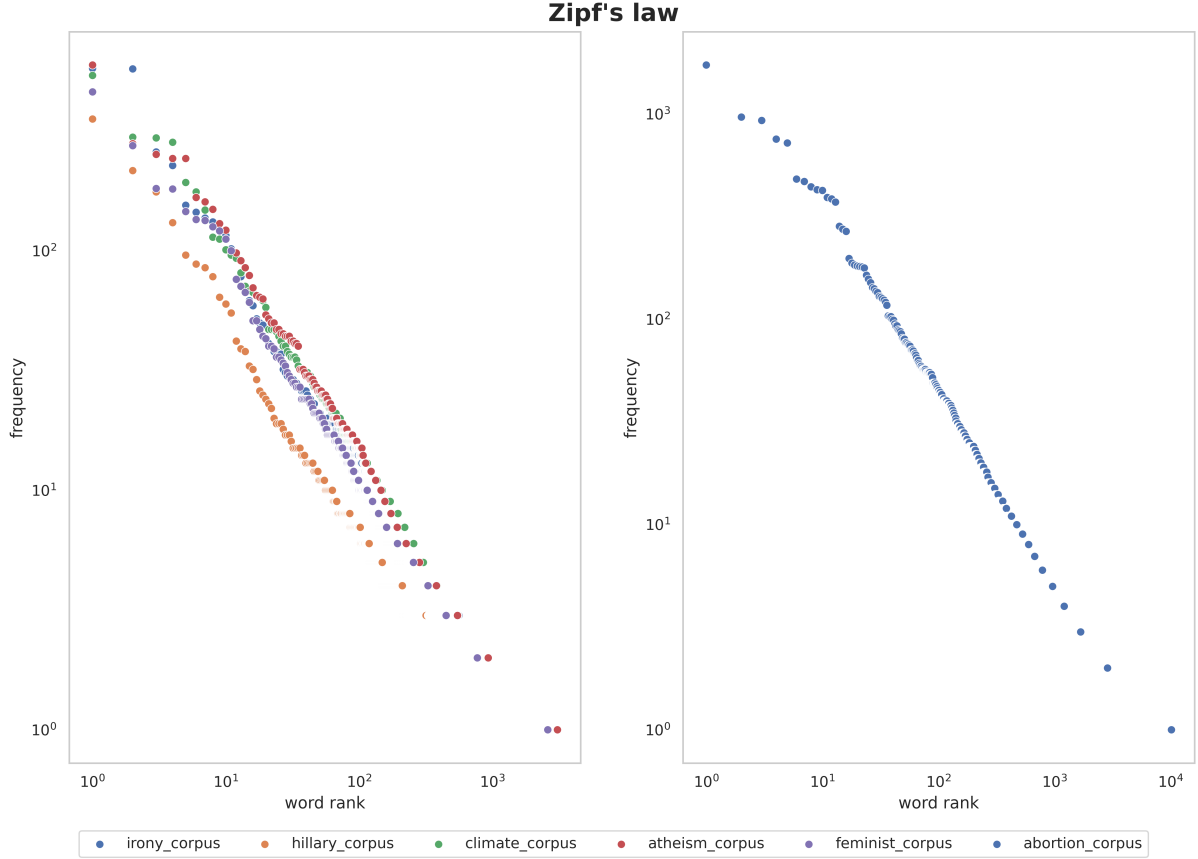


Figure 1: Zipf's law

Furthermore, we do not expect links to contain any meaningful knowledge, so we transformed all links in our corpus to "@link", to gather them all into one token. Finally, if tweets contained three or more occurrences of the following punctuation marks; (! , . ?), it was tokenised to $3 \times$ the first punctuation mark in the token. I.e., "!!!!!" is converted to "!!!!".

3 Annotation

After an examination of the research paper "Irony Detection in English Tweets" from the SemEval workshop, we took a random sample from the irony training set (`train_text.txt`) consisting of 100 tweets. Without interaction, we each annotated all 100 tweets. To do so, we wrote a script (`annotate.py`) which displayed a tweet in our terminal and from there we had to decide if the tweet was ironic or not. To investigate the quality of our individual annotation, we calculated the inter-annotator agreement between every possible combination of pairs in the group, together with the gold/true labels. These numbers are represented in table 2.

Since our average agreement with the gold labels was 45.9%, we inquired into the individual annotations, to find out where we disagreed. To find out which tweets we disagreed on, we summed over all the rows of our data frame, consisting of the individual annotations and the gold labels. Next we returned all the tweets, and the corresponding gold labels, where the sum was equal to three, since a sum of three meant that half of us, including the gold label, disagreed with the other half. After further investigation we established that most of the disagreement was caused by lack of context in the tweets.

Annotator 1	Annotator 2	Kappa Pairwise
Marie	Frida	0.376
Marie	Magnus	0.408
Marie	Viggo	0.362
Marie	Gustav	0.333
Frida	Magnus	0.601
Frida	Viggo	0.606
Frida	Gustav	0.470
Magnus	Viggo	0.615
Magnus	Gustav	0.440
Viggo	Gustav	0.486
Marie	Gold	0.517
Frida	Gold	0.364
Magnus	Gold	0.545
Viggo	Gold	0.465
Gustav	Gold	0.403

Table 2: Inter Annotator Agreement

A big portion of the tweets in the data set are parts of tweet threads, meaning that the rest of the conversation is not provided. This also applied to shared photos on Twitter, where we are only given the caption as a tweet. Lack of context are shown in the following examples:

- 1) *You're any moment, every second, | Every scent and every flower | As a girl.*
- 2) *#PhatSexyAss #Picnic Picnic Porked Tiffany*

Tweets deviate from natural language in many ways in how they are constructed (shortness, hash-tags, lack of context, etc.) and often include misspellings and grammatical errors. This all contributed to the disagreement between annotators.

4 Classification

For our classification we decided to use two different classifiers from the `scikit-learn` library. For the binary classification task we used the `SDGClassifier`. The Stochastic Gradient Descent fits linear classifiers and regressors under convex loss functions. SGD is efficient and easy to implement, but sensitive to feature scaling. After experimenting with tweaking different parameters, we found the best accuracy scores with `learning_rate` set to 'constant', and the `n_iter_no_change` was set to 500 i.e., the number of iterations without improvement of the model before stopping the fitting.

For the multi class classification we started off by trying out the `SGDClassifier`, but the majority of tweets were labelled the same. The `MLPClassifier` provided us with better results for the multi class data sets. With the Multi-layer Perceptron classifier we changed the parameter `Learning_rate` to 'adaptive' and `activation` to in the hidden layer 'tanh', the hyperbolic tan function. For the multi class classification we trained the model on the feminist training set. That model was then tested on the test sets of all 5 stance topics.

The precision score, the recall score, the f1-score and support is reported in Table 3, for the binary class data and the multi class data.

As an attempt to improve the model’s prediction performance we used self-training. We constructed an augmented data set to perform the self-training on, consisting of tweets from the original training data set and tweets from the entire `offensive` data set. The tweets we chose to merge into the augmented data set were chosen based on their perplexity score - a score indicating how confused a model is based on what it sees and is defined as

$$PPL = p(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

The perplexity scores were obtained by training a bi-gram language model on our training corpus, using Lidstone smoothing, with $\gamma = 0.2$. The γ -value was optimised by empirical testing. Ideally we would have used the Kneser-Ney smoothing method, as it is considered a better smoothing method than Lidstone. However, we experienced some issues computing perplexity scores with Kneser-Ney, and therefore we resorted to the Lidstone smoothing method. In hindsight, our problems with Kneser-Ney method could have been due to the fact that our vocabulary contained a lot of infrequent words.

We then used the classifier trained on the irony training set to predict labels for the offensive set. Afterwards we selected the offensive tweets with the lowest perplexity scores and mixed them with the irony training set. We tried mixing in different amounts of the offensive tweets to see which number gave the best result. Using the model trained on the augmented data set, we predicted labels for the irony validation set to test the accuracy of the models.

The best results were obtained by merging 1500 tweets from `offensive` with the irony training set for the binary classification, and 300 tweets from `offensive` with the feminist training set for the multi class classification. For the binary classification, using the augmented data set caused a 4.85% increase in our models f1-score. For the multi class model, an improvement of 3.03% was reached on average. The precision scores, the recall scores, the f1-scores and support for the augmented data sets are reported in Table 3.

	Irony	Abortion	Atheism	Climate	Feminist	Hillary
No data augmentation						
Acc_score	0.642	0.571	0.550	0.266	0.575	0.478
Precision	0.542	0.382	0.398	0.333	0.464	0.412
Recall	0.617	0.376	0.325	0.356	0.432	0.374
F1-score	0.577	0.368	0.339	0.210	0.415	0.349
Data augmentation						
Acc_score	0.666	0.614	0.577	0.231	0.604	0.502
Precision	0.569	0.429	0.376	0.338	0.496	0.365
Recall	0.646	0.403	0.321	0.402	0.446	0.356
F1-Score	0.605	0.409	0.332	0.201	0.444	0.346

Table 3: Classification results

5 Conclusion and Future Work

Through working on this project, we have gained insight into some of the many parts of Natural Language Processing. We have experienced how human language can vary a lot, and it can be hard to evaluate the meaning of text without context. There are many parts of the NLP process, where methods are important. How data sets are annotated, what preprocessing is done before training a model, which model is chosen, the parameters of the model, amongst others. We have learned the basics of how to organise labelling of data sets and evaluate the agreement between annotators. We have also learned how to think about some of the decisions that have to be made when developing a tokeniser and preprocessing data. We also learned some metrics to evaluate the results of a classifier by. That being said, NLP is a broad field, and we have only touched on the surface of what it has to offer. It is also very active, and there is constant development in the technologies and discussion on how cognitive psychology can be adapted into something a computer can understand, process and calculate.

In future work on this topic or a related one, it could be interesting to use a more sophisticated language model than the ones we used in this project. Our prediction models only use a bag of words as the basis for the models. A more sophisticated model could look at bigrams and/or higher order n-grams. One could also look at more advanced syntax parsing, or lemmatisation - a tool from NLP which does full morphological analysis to accurately identify the lemma for each word; for example, from "produced", the lemma is "produce", but the stem is "produc-". (Manning et al., 2009)

We had some issues with calculation of perplexities because the library NLTK uses closed language models for Kneyser-Ney and Witten-Bell. A closed model does not assign a probability to words that are not in its vocabulary. This caused issues with tweets, as there will almost always be words in a test corpus that were not present in the training corpus. In future work, we could also use a library better suited for more advanced NLP tasks.

The final models we have trained on and tested were an SDG classifier for the irony data set and an MLP classifier for the stance data sets. We made an augmented data set using a classifier to label the offensive data set and trained on that as well. For irony we trained one model solely on the irony data set and one model with the 1500 tweets from offensive with the lowest perplexities added. For stance we trained one model solely on the feminist data set and one model with the 300 tweets from offensive with the lowest perplexities added in.

For the irony data set, we got an accuracy score of 0.642 without data augmentation and 0.666 with data augmentation. For the stance data sets, the abortion data set had the biggest improvement. The accuracy score without data augmentation was 0.571 and the score with data augmentation was 0.614. Apart from the climate data set, the accuracies for all data sets were improved by using data augmentation. The precision, recall and F1-Score was also improved for most data sets using data augmentation.

In conclusion, we have trained models that can make a reasonable guess about the presence of irony or the stance taken towards a topic. We have successfully used data augmentation to improve the accuracy of said models. The best performing model had an accuracy of 0.666 on the test set, which certainly could be improved with more work. One major hindrance has been the limited capabilities of the NLTK library, that uses closed language models for its more advanced models. If we were to do continued research on this topic, we would use a different library, and possibly look into more advanced language models as mentioned.

6 Disclosure Statement

The gitlog contains mostly commits from Gustav Kristensen. This is because we worked collaboratively in DeepNote, where he was logged into git in the terminal, so all commits from DeepNote are from his account. There was no imbalance in the workload unlike what it seems like from the gitlog.

References

- Manning, C., Raghavan, P., & Schuetze, H. (2009). Introduction to Information Retrieval, 581.
- Van Hee, C., Lefever, E., & Hoste, V. (2018). SemEval-2018 Task 3: Irony Detection in English Tweets. *Proceedings of The 12th International Workshop on Semantic Evaluation*, 39–50. <https://doi.org/10.18653/v1/S18-1005>